Laboratorio di Algoritmi e strutture dati

Anno 2021

Valentino Bocchetti

Contents

1 Introduzione al corso **1**

2 Argomenti del corso **1**

3 Gestione del corso (prove intercorso) **1**

4 Esame **3**

5 Lezione del 11-03 **3**

5.1 Struttura modulare di un programma 3

5.1.1 Struttura di un progetto 4

6 Lezione del 12-03 **5**

6.1 Tipi fondamentali, Puntatori e Riferimenti 5

6.1.1 Dichiarazione di variabili 5

6.1.2 Tipi fondamentali in c++ 5

6.1.3 Tipi const 6

6.1.4 Puntatori 6

6.1.5 Void pointer 7

6.1.6 Riferimenti 7

6.1.7 Lvalue & Rvalue 9

1 Introduzione al corso

`Ricevimento` → Lunedì ore 16-18 (Teams).

`Email` → fabio.mogavero@unina.it

Link al download del libro di Data Structure & Algorithm Analysis → clicca per il download

2 Argomenti del corso

- Elementi di Linguaggio C++
- Tipi di dato astratto e relative implementazioni in C++
- Progettazione di una libreria contenitore di dati
- Strutture dati elementari (vettori, liste, pile, code)
- Alberi binari di ricerca e iteratori sui dati
- Matrici e grafi

3 Gestione del corso (prove intercorso)

- Esercizio Vettori e liste (26 Marzo - 11 Aprile);
- Esercizio Pile e Code (9 Aprile - 25 Aprile);
- Esercizio Alberi Binari di Ricerca (23 Aprile - 9 Maggio);
- Esercizio Iteratori su Alberi (7 Maggio - 23 Maggio);
- Esercizio su Matrici (21 Maggio - 6 Giugno);
- Esercizio sui Grafi (4 Giugno - 20 Giugno).

Per la consegna:

- Invio file .zip o tar.gz alla mail del prof con oggetto "Consegna Esercizio x (Cognome-Nome-Matricola)"
- Formato File: Cognome-Nome-Matricola(Esercizio x).zip *o* Cognome-Nome-Matricola.tar.gz
- Scadenza improrogabili → ore 23:59 dell'ultimo giorno previsto per l'esercizio

4 Esame

Esame in presenza:

- Prova al calcolatore + discussione orale

Per chi consegna tutti gli elaborati si ha la possibilità di accedere all'esame semplificato (reimplementazioni, implementazione di una `vecchia/nuova` funzionalità di una libreria tra quelle implementate)

Per chi non consegna tutti gli elaborati si è obbligati ad accedere all'esame completo (implementazione da zero di una libreria)

Prerogative per entrambe le modalità è l'obbligo della consegna di tutte le librerie con data ultima precedente alla data di esame a cui si vuole accedere

Nel caso di modalità telematica nel caso dell'esame semplificato si svolgerà solo la parte orale

5 Lezione del 11-03

5.1 Struttura modulare di un programma

Primo esempio (main.cpp)

```
#include <iostream> // libreria standard c++ I/O
// dichiarazione dello spazio dei tipi
//(accesso alle funzionalità senza la specificazione "std::")
using namespace std;
int main(){
    cout << "Hello World!" << endl;
}
```

Di seguito invece il `build.sh` (script bash)

```
#!/bin/bash
g++ [-O3] -o main main.cpp
```

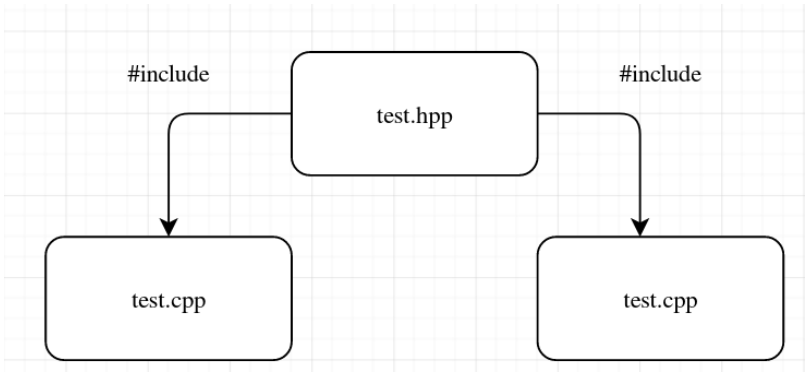
Dove:

- g++ → compilatore GNU;
- `[-O3]` → parametri di ottimizzazione opzionale;
- -o main → `output/file eseguibile`;
- main.cpp → codice sorgente

Nel momento in cui ci siano più file da compilare vanno aggiunti manualmente (l'ordine è indefferente)

5.1.1 Struttura di un progetto

Nel caso di più file si avrà un file principale più altri che eseguiranno effettivamente le operazioni:



test.hpp → file header. Si presuppone che abbia delle macro iniziali e finali che viene letto dal precompilatore

```
// main.cpp
#include <iostream>
#include "test.hpp"
using namespace std;
int main(){
    cout << "Chiamata a funzione";
    test();
}
```

```
// test.hpp
// Le macro vengono risolge dal preprocessore
#ifndef TEST_HPP
#define TEST_HPP
void test();
#endif

// test.cpp
#include <iostream>
#include "test.hpp"
void test(){
    std::cout<<"OK!"<< endl;
}
```

6 Lezione del 12-03

6.1 Tipi fondamentali, Puntatori e Riferimenti

6.1.1 Dichiarazione di variabili

`type varname [= defaul value]`

`type varname [numero elementi] [{` $\text{val}_0, \ldots, \text{val}_n$ `}]`

6.1.2 Tipi fondamentali in c++

- Bool (`true/false`);
- `[unsigned]` char;
- `[unsigned]` short;
- `[unsigned]` int;
- `[unsigned]` long int;
- `[unsigned]` long long int;
- float;
- double;
- long double

In alcuni compilatori è possibile delle abbrevazioni per tipi unsigned int e long del tipo:

- uint;
- ulong;

6.1.3 Tipi const

```
const type varname = value;
```

Ritornano utili anche nel passaggio dei parametri nelle chiamate a funzione

6.1.4 Puntatori

```
type* varname [=default address]
```

es.

```
#include <iostream>
using namespace std;
int main(){
    char c = 'a';
    char* p = &c;
    cout << "Il valore di *p è "<< *p << endl;
    p= nullptr; //corrisponde alla macro NULL del linguaggio C
    cout << p << endl;
}
```

```
Il valore di *p è a
```

```
#include <iostream>
using namespace std;
int main(){
    const char c = 'a';
    char* p = &c; // ERRORE
    const char* p = &c; // CORRETTO
    cout << *p << endl;
}
```

6.1.5 Void pointer

```
void* varname [= ...]
```

Un void pointer è un puntatore che non è associato a nessun tipo specifico. Puo mantenere un indirizzo di ogni tipo e può essere typecasted a ogni tipo.

Indefinire un puntatore in cui non è noto a priori il tipo di dato a cui si vuole puntare

```
#include <iostream>
using namespace std;
int main(){
    char c = 'a';
    void* p = &c;
    //cout << *p << endl; // ERRORE
    cout << *((char*)p) << endl; // CORRETTO
    cout << *(static_cast<char*>(p)) << endl; // MODO MIGLIORE
}
```

6.1.6 Riferimenti

```
type & varname = ...
```

```
#include <iostream>
using namespace std;
int main(){
    uint d = 7;
    uint & e = d;
    e++;
    cout << "Il valore di d è " << d;
}
```

```
Il valore di d è 8
```

```
#include <iostream>
using namespace std;
int main(){
    uint d = 7;
    uint e = d;
    e++;
```

```
    cout << "Il valore di d è " << d;
}
```

```
Il valore di d è 7
```

Con i puntatori invece:

```
#include <iostream>
using namespace std;
int main(){
    uint d = 7;
    uint* e = &d;
    (*e)++;
    cout << "Il valore di d è " << d;
}
```

```
Il valore di d è 8
```

```
int & i = 1; // ERRORE
const int & i = 1; // CORRETTO
```

6.1.7 Lvalue & Rvalue

Differenza tra Lvalue e Rvalue

In C++ un Lvalue è qualcosa che punta a una specifica locazione di memoria. Un Rvalue invece è qualcosa che non punta da nessuna parte.

In generale gli Rvalues sono temporanei e di vita breve, mentre gli Lvalues hanno una vita lunga tanto quanto quella della variabile.

```
#include <iostream>
using namespace std;
string f(){return string ("Ciao");};
int main(){
    string var = f();
    string stringa1 = move(f());
    string && stringa2 = f(); // Riferimento Rvalue
    string stringa3 = move(stringa2); // Spostiamo il valore di stringa2 in stringa3 se
    cout << var<< endl;
    cout << stringa1<< endl;
    cout << stringa2<< endl;
    cout << stringa3;
}
```

Ciao
Ciao
Ciao